Cut Cross-Entropy

Cut Your Losses in Large-Vocabulary Language Models

Erik Wijmans · Brody Huval · Alexander Hertzberg · Vladlen Koltun · Philipp Krähenbühl · Apple

PROBLEM

Large-vocabulary CE

Cross-entropy строит score для каждой пары «training token × vocabulary item».

PAPER QUESTION

Нужна ли полная logit matrix?

Авторы исследуют classifier head и вычисление loss, а не transformer backbone.

PROPOSAL

CCE

Correct-token logit вычисляется отдельно, log-sum-exp — on the fly.

Где находится Cross-Entropy?

01
Tokens
Последовательность из vocabulary V.

02
Backbone f
Строит feature vector контекста.

03
Embedding E_i
$E_i = f(x_{<i}) \in \mathbb{R}^D$.

04
Classifier C
$C \in \mathbb{R}^{D \times |V|}$.

05
Softmax / CE
Вероятность правильного next token.

$$P(x_i \mid x_{<i}) = \mathrm{softmax}_{x_i}\left(C^\top E_i\right)$$

Paper wording: classifier projects a D-dimensional feature vector into the output space of vocabulary V. Наше пояснение: для Transformer E_i — hidden state после backbone.

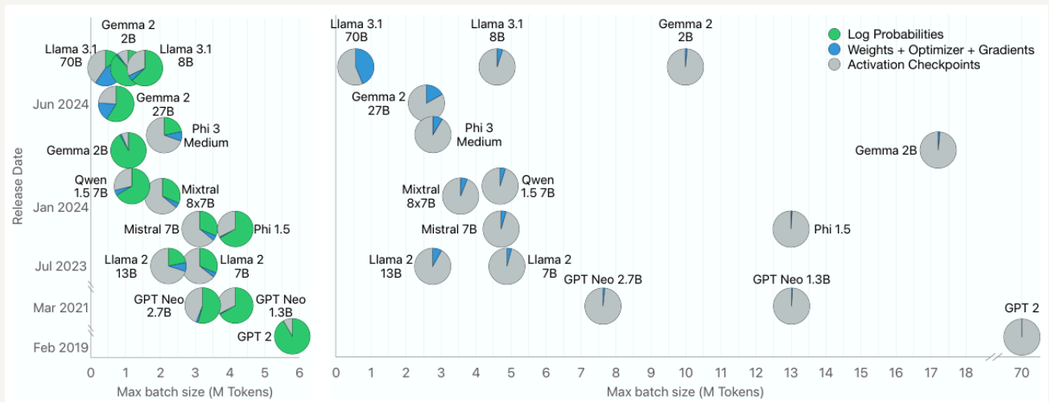
На training появляется матрица $|V| \times N$

PAPER SETUP

Teacher forcing

Backbone эффективно вычисляет $E_1, \ldots, E_N$ параллельно, а classifier выдаёт scores для каждого training token по всему vocabulary.

$$E = [E_1 \ldots E_N], \qquad A = C^\top E \in \mathbb{R}^{|V| \times N}$$



(a) Regular cross-entropy (b) Cut cross-entropy (ours)

Paper §3.2: inference требует distribution для одного token at a time; training оптимизирует все next-token predictions совместно. **§3.2, стр. 4:** для Gemma 2 2B log-probabilities достигают 89% training memory в их setup.

Loss раскладывается на два вычисления

$$\ell = \left(C^\top E\right)_x - \log \sum_j \exp\left(C_j^\top E\right)$$

1. score только ground-truth labels **2.** log-sum-exp по всему vocabulary

Наша сноска: это обычный log-softmax правильного класса: $\log p_y = z_y - \log \sum_j e^{z_j}$. Paper максимизирует log-likelihood; стандартный CE меняет знак.

Indexed matrix multiplication

Linear log-sum-exp

Indexed matmul: вычислить только correct-token logits

Полный classifier output

01 Для каждого E_i вычислить scores для всех $|V|$ columns C_j.

02 Потом выбрать score $C_{x_i}^{\top} E_i$.

Paper §4.1

01 Прочитать target id x_i.

02 Загрузить только column C_{x_i} и embedding E_i.

03 Сразу выполнить dot product.

Forward indexed matmul использует on-chip SRAM и не аллоцирует отдельную $|V| \times N$ матрицу.

Paper §4.1: операции выполняются blockwise для эффективности cache; Algorithm 1 / Figure 2a описывают access pattern.

Linear-log-sum-exp: block matmul → block LSE → update

Regular path

01 GEMM строит $A = C^\top E$.

02 Blocks результата пишутся в global memory.

03 Отдельная reduction читает A и считает LSE.

CCE forward

01 Block matmul $C_m^\top E_n$ в SRAM.

02 Сразу LSE вдоль vocabulary-range блока.

03 Partial LSE обновляет общий LSE_n.

04 Полный A не материализуется.

Paper §4.2: CCE использует ту же blocking/parallelization strategy, что и efficient matmul. **Наша сноска:** stable online LSE хранит running scale/max и сумму, поэтому промежуточные logits можно отбросить после reduction.

Backward: формула и приближение

ALGEBRA

Восстановить probabilities

Оси PyTorch на этом слайде: E — N×D, C — V×D. Y — one-hot targets, P — N×V.
Для каждого блока повторно вычисляются logits. Сохранённый LSE используется для нормализации.

$$P_{ij} = \exp(Z_{ij} - s_i)$$

$$G = (P - Y)/N, \quad \nabla_E = GC, \quad \nabla_C = G^\top E$$

IMPLEMENTATION

Filtering — отдельная настройка

В выбранном commit BF16 auto threshold = 2^{-12}. Проверка блока находится в cce_backward.py. Это не минимальное представимое число BF16 и не доказательство нулевой ошибки.

Сравниваем default, no-filter и exact; публикуем ошибки loss и обоих gradients.

Формулы: REPORT.md §5, при N валидных токенах. Код: upstream 3de376c, cce_backward.py и utils.py. Измерения: results/main/correctness.json.

Один objective — разный dataflow

Standard linear + CE

01 Вычислить $A = C^\top E$.

02 Материализовать $|V| \times N$ output.

03 Получить target log-probabilities / CE.

Cut Cross-Entropy

01 Correct-token term: indexed matmul.

02 Normalizer: blockwise linear-LSE.

03 Backward: recompute + gradient filtering.

Ключевой claim paper: нет необходимости материализовать полную logit matrix в GPU global memory.

Важно для защиты: CCE не уменьшает vocabulary и не меняет training objective. Он меняет реализацию classifier-head cross-entropy.

Table 1: та же метрика памяти

Величина	Статья, «MB»	HW2, MiB	Статус HW2
CCE, loss	1	1.040	измерено
CCE без filtering/sorting, loss	0.09	0.094	измерено
CCE, loss+gradient	1164	1164.269	измерено
CCE без filtering/sorting, loss+gradient	1162	1162.316	измерено
Lower bound: BF16 ∇E + ∇C	1161	1161.000	расчёт
Baseline, loss	24000	24000.001	экстраполяция*
Baseline, loss+gradient	28000	28000.001	экстраполяция*

Измеренная память CCE близка к округлённым значениям Table 1. Extra peak = peak allocated − allocated before; единицы — 2^{20} bytes.

* Baseline: экстраполяция из N512,1024,2048; N8192 не запускался. Это не наблюдённый OOM и не измеренное отношение baseline/CCE. Источники: отчёт §10.8, Table 1 (PDF стр. 8), upstream benchmark/memory.py; расчёт из JSON. Состав batch и протоколы различаются.

Что мы воспроизводим

ОСНОВНОЙ ЭКСПЕРИМЕНТ

Gemma classifier head

Реальные hidden activations и output weights Gemma 2 2B. Alpaca: 52 примера, 8192 валидные позиции; V=256000, D=2304, softcap=30.

CORRECTNESS

Loss, ∇E и ∇C

Gemma: 5 checks против FP32 на N=32 и полном V. Синтетика: 60 checks против FP64. Допуски: |Δloss| < 0.05, relL2 < 5%.

ДОПОЛНИТЕЛЬНЫЕ ПРОВЕРКИ

Shapes и profiler

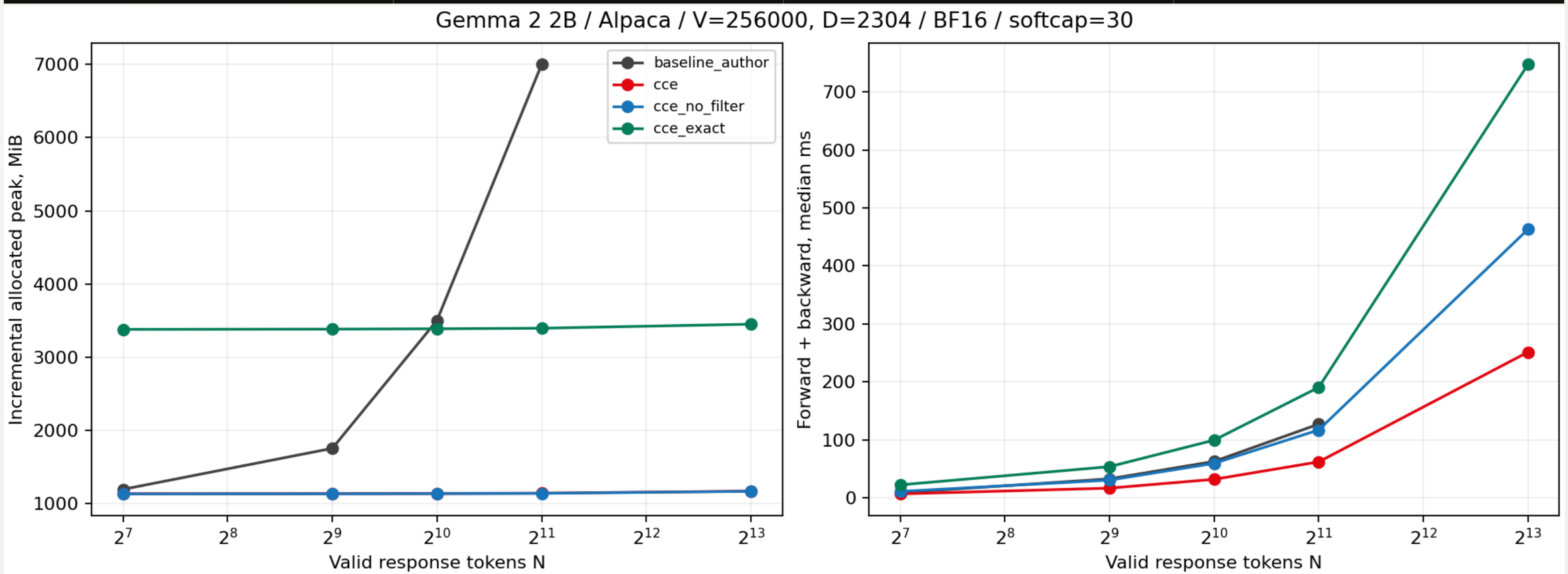
Синтетическая сетка: 11 shapes, 88 строк. Отдельный backward, повторные timings, варианты filtering/accumulation. Всего 15 profiler-трасс.

5070 Ti	2.11.0	3.6.0	3de376c
16 GB · Windows	PyTorch · CUDA 12.8	Triton Windows	upstream commit · BF16

5 warmup + 30 CUDA-event samples на точку; TF32 off. Extra peak = peak allocated − allocated before operation. Backbone не входит в benchmark. Запуск: notebook, §§10–12; pinned revisions — отчёт.

Gemma: classifier-head forward+backward

−67.6%	1.97×	38	5/5
extra peak: 3500.00 → 1132.73 MiB (3.09×)	head: 62.75 → 31.88 ms	измеренных строк; 2 planned skips	checks на N=32 против FP32



Числа сверху: N=1024,V=256000,D=2304, BF16, softcap=30, forward+backward. 52 примера, 8192 валидные позиции. Backbone исключён. Baseline N=8192 не запускался по заранее заданному memory guard; OOM не наблюдался. Протокол Gemma; results/gemma2/.

Выводы в пределах измерений

MEASURED

Gemma classifier head

N=1024, V=256000, D=2304, softcap=30: extra peak 3500.00 → 1132.73 MiB. Это peak сверх живых входов; gradients включены.

DEPENDS ON SHAPE

Ускорение не универсально

N1024/V65536/D2048: 13.69 ms CCE против 12.18 ms baseline (+12.3%). Направление согласуется с механизмом параллелизма из Appendix C.2; контрольные методы отличаются.

LIMITS

Границы проверки

Сравниваются режимы upstream, а не один изолированный threshold. Exact — название preset с FP32 accumulation, не отсутствие округления. LLM convergence и perplexity не проверялись.

Память согласуется с Table 1. Runtime согласуется с механизмами §5.2 и App. C.2; локальная причинность не проверена. Softmax sparsity и torch.compile остаются пробелами.

Численный пример: почему не нужен полный softmax output

$$L = -z_y + \log \sum_j e^{z_j}$$

Пример: scores [2, 1, 0], target=0. LSE ≈ 2.407606; loss ≈ 0.407606; p(target) ≈ 0.665241.
Для loss достаточно target score и normalizer.

Не из paper: pedagogical example / derivation для вопросов после доклада.

Stable / online log-sum-exp

$$\mathrm{LSE}(z) = m + \log \sum_j e^{z_j - m}, \qquad m = \max_j z_j$$

Partial reductions можно объединять без сохранения полного вектора logits.

Paper использует online/blockwise LSE; эта алгебраическая расшифровка — наша поясняющая выкладка.

Как измеряли

Память

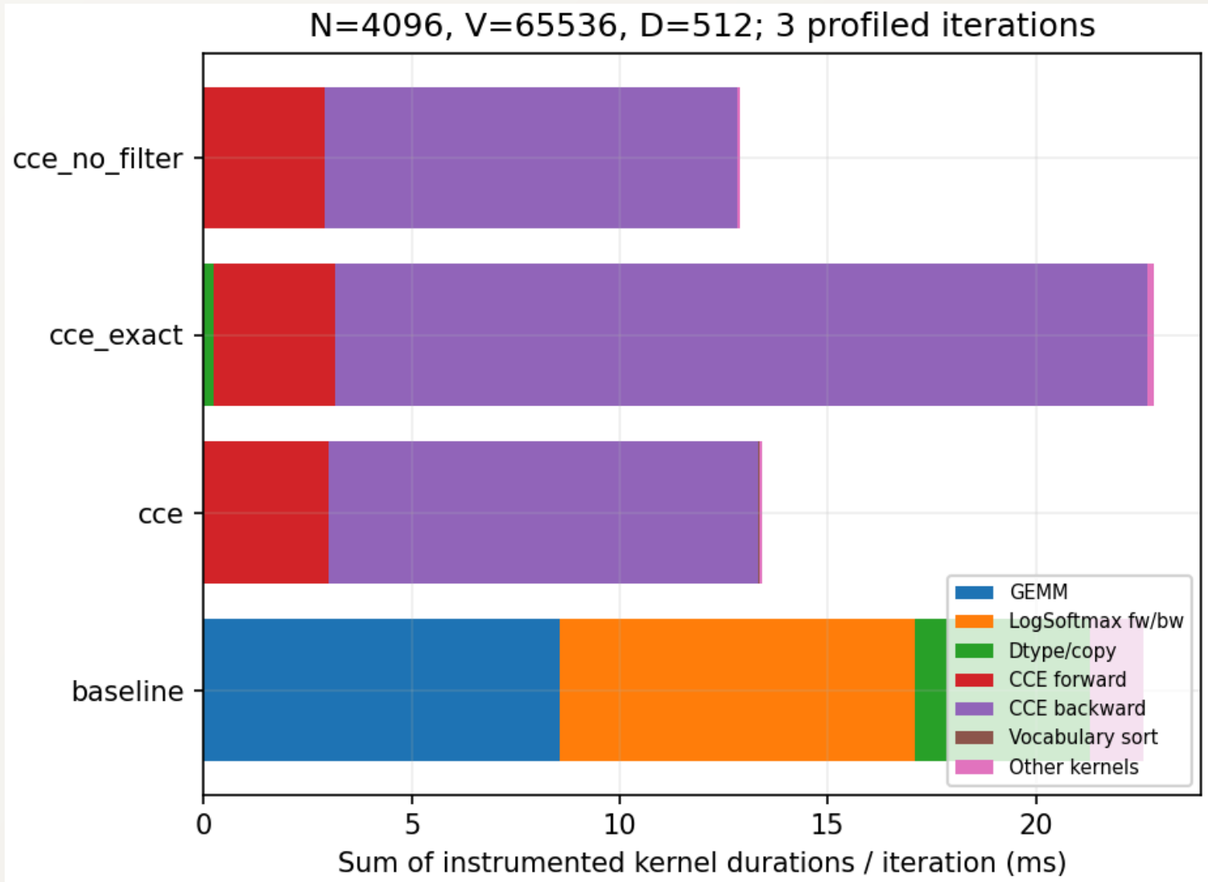
- Входы созданы до измерения.
- После warmup: loss удалён, grads=None.
- Сохранён baseline allocated.
- Reset peak → один проход → synchronize.
- Extra = peak − baseline; gradients включены.

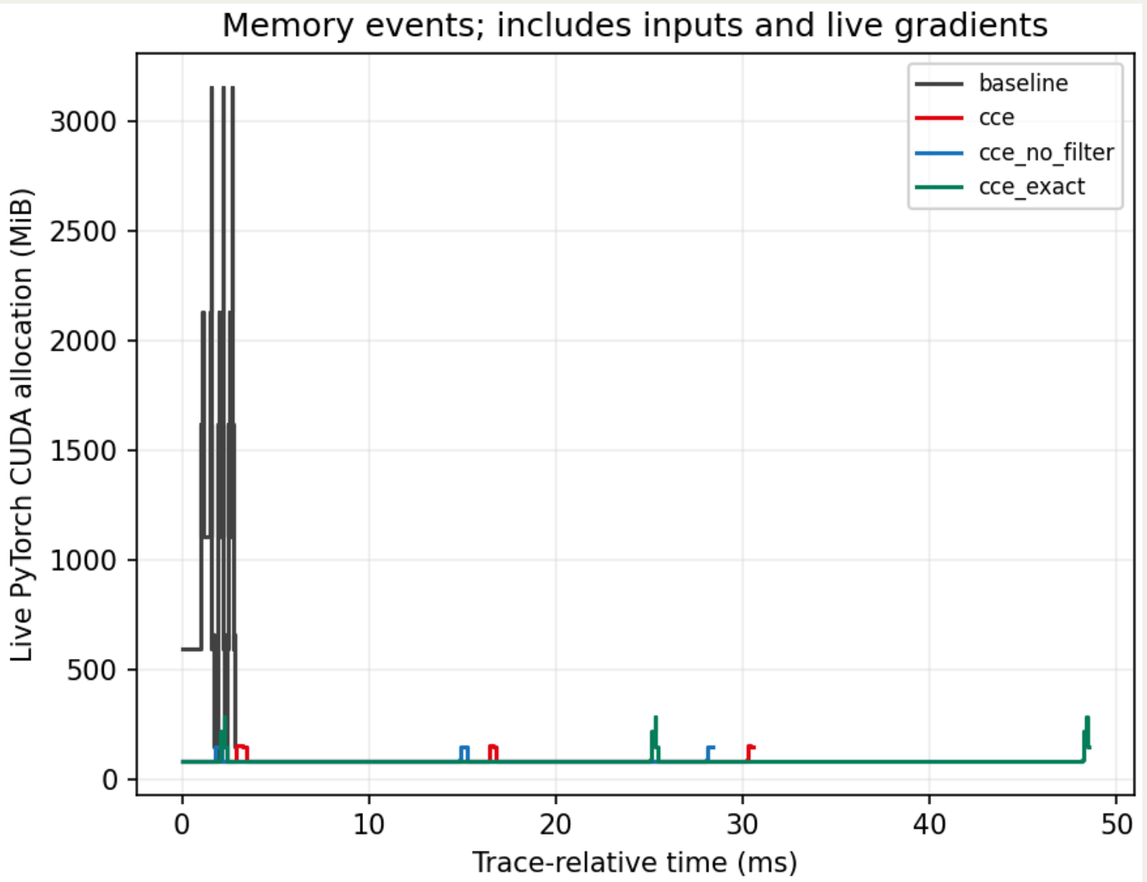
Время

- 5 warmup; компиляция вне samples.
- 30 CUDA-event samples; median и p10–p90.
- Порядок методов чередуется между shapes.
- Forward с autograd; total включает backward.
- Backward-only: свежий graph до start event.

Точная среда: results/main/environment.json. Все samples: measurements.json. Дополнительный backward: results/diagnostics/diagnostics.json. TF32 disabled; BF16 inputs.

Что видно в profiler



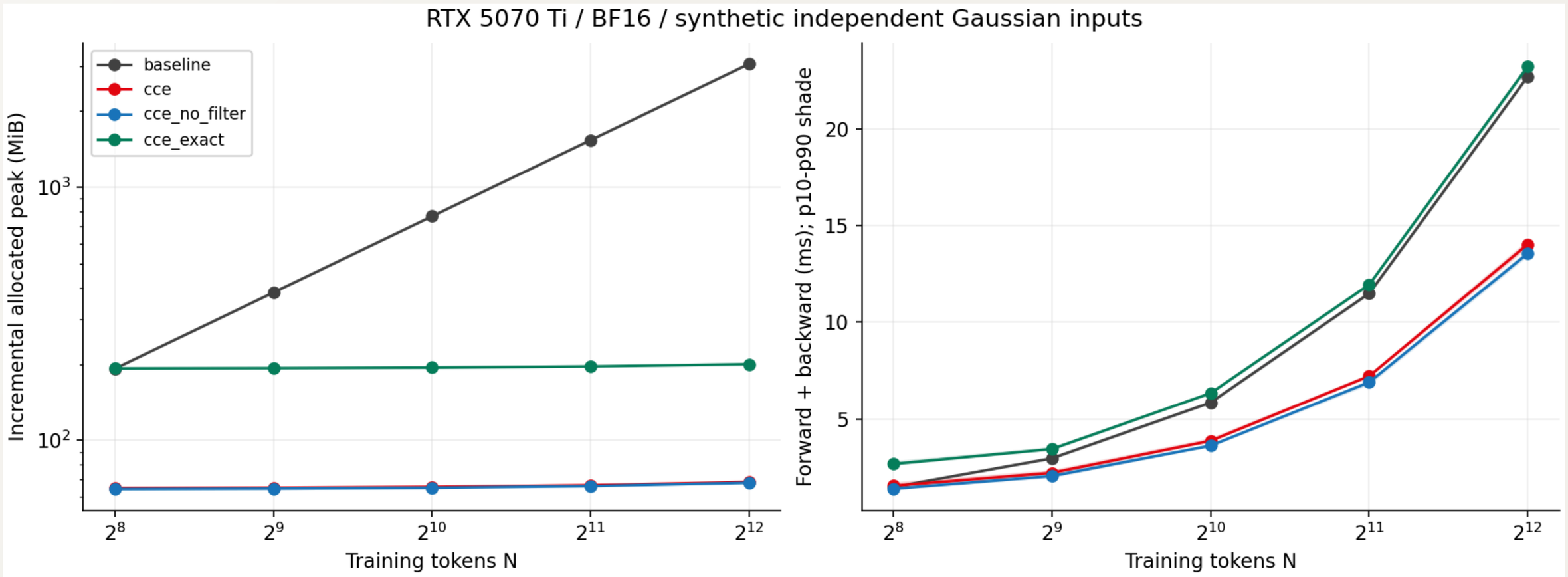


Baseline: allocations по 1 GiB. CCE default: крупнейший allocation 64 MiB. Sorting kernels есть в default и отсутствуют в no-filter.

N=4096,V=65536,D=512. Три iterations под profiler, не обычный latency benchmark. Трассы: results/profiles/*.trace.json.gz; разбор профилирования. Доля skipped tiles и DRAM counters не измерены.

Синтетика: дополнительный peak и время

−97.8%	1.62×	60/60	0.822%
extra peak: 3072.00 → 68.53 MiB	быстрее: 22.71 → 14.03 ms	проверок в допусках	max relL2 ∇E для CCE vs FP64



Числа сверху: N=4096, V=65536, D=512, BF16, forward+backward. График: V=65536, D=512, sweep N. Полный allocated peak: 3156.28 → 152.81 MiB (20.65×). results/main/measurements.json